Aspect-Based Sentiment Analysis on TripAdvisor Hotel Reviews

UE24CS352A Machine Learning – Mini-Project Write-up

Team: Shikshith V (PES1UG24AM400) and Sateesh (PES1UG24AM397) | **Code:** github.com/Shikshith05/ML-Miniproject

1. Problem Statement

On TripAdvisor, a single star rating does not tell us why guests liked or disliked a hotel. In this project we read the text of a review and predict the **overall star rating** and the star ratings of **five aspects**: value, rooms, location, cleanliness and sleep quality (1 to 5 stars each). We then turn the stars into **polarity**: below 3 is negative, 3 is neutral and above 3 is positive. Our project is based on the Stanford CS229 report *Aspect-based Sentiment Analysis on Hotel Reviews* [1]. We wanted to (i) compare Linear SVM, Naive Bayes and Ridge regression, (ii) see how they do against a simple baseline that always guesses 5 stars, (iii) check whether training on an aspect's own ratings works better than training only on overall ratings, and (iv) make a small live demo. The paper also predicts Service and Business Service, but our dataset does not have those ratings, so we only predict the overall rating and five aspects.

2. Dataset Details

We used the Hugging Face dataset `jniimi/tripadvisor-review-rating` [2]. It is a cleaned, English-only collection of TripAdvisor hotel reviews, based on data from Li, Ott and Cardie (2013). Each row has the review title and body (the `review` column), an `overall` rating, and ratings for cleanliness, value, location, rooms and sleep quality. The full file has about 201,000 reviews. To keep things fast on a laptop we randomly picked **30,000** of them (seed 42), removed reviews with 20 characters or fewer, and treated any rating outside 1 to 5 as missing, so that no model learns from a rating that does not exist. After this, every review had all six ratings. We then split the data into **20,000 training and 10,000 test** reviews (2/3 and 1/3, as in the paper). The ratings are **very skewed**: 5 stars is the most common rating for every target, and a model that always says 5 stars already gets 41 to 64% accuracy (63.8% for location). Because accuracy alone can be misleading, we also report macro-F1, within-1-star accuracy and this baseline.

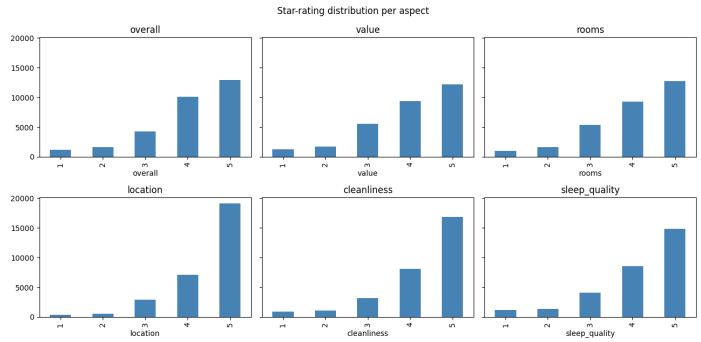


Figure 1. Star-rating distribution for each target (30,000 sampled reviews).

3. Approach

Features. We change each review into counts of word phrases of 1 to 3 words (`CountVectorizer`). We keep phrases that appear in at least 2 reviews but in no more than 20% of them, up to 300,000 features, and the vocabulary is built from the training reviews only. We did not remove stop words, because words like "not" matter ("not good" should stay together). We also did not use TF-IDF. The paper found that in hotel reviews the sentiment words ("great", "dirty") show up in almost every review, so TF-IDF would reduce their weight. We followed that choice and did not test TF-IDF ourselves.

Models. For each of the six targets we trained three models: a Linear SVM (`C=0.1`, with balanced class weights so the rare 1 and 2 star reviews are not ignored), Multinomial Naive Bayes (`alpha=0.5`) and Ridge regression (`alpha=5`), whose output we round to the nearest star from 1 to 5. That gives 18 models. We used fixed settings and did not tune them.

Metrics. We measure exact star accuracy, within-1-star accuracy, macro-F1 and polarity accuracy, and compare each with the always-5-stars baseline.

Main experiment. We also trained one Linear SVM using only the overall ratings, used it to predict each aspect, and compared it with the SVM trained on that aspect's own ratings.

4. Implementation Overview

Everything is in one notebook, `absa_hotel_reviews.ipynb`. It loads and cleans the data, plots the ratings, splits the data, builds the features, trains the 18 models and scores them against the baseline. It also has the confusion matrix, the aspect-vs-overall comparison, the top phrases for each aspect, the saved models (joblib) and a summary cell. The function `predict_review(text)` gives the stars and polarity for all six targets, and we use it for the live demo. The tables and plots are saved in the `outputs/` folder, and all numbers in this report come from those saved tables. Setup and run steps are in the README.

5. Results

	Star accuracy				Polarity accuracy			
Target	**Base**	**SVM**	**NB**	**Ridge**	**Base**	**SVM**	**NB**	**Ridge**
Overall	0.432	0.618	**0.651**	0.560	0.764	0.839	**0.854**	0.824
Value	0.409	0.485	**0.525**	0.445	0.724	0.756	**0.775**	0.757
Rooms	0.425	0.536	**0.560**	0.499	0.732	0.779	**0.787**	0.779
Location	0.638	0.626	**0.646**	0.557	0.876	0.861	**0.862**	0.861
Cleanliness	0.560	0.589	**0.626**	0.548	0.826	0.845	**0.850**	0.839
Sleep quality	0.496	0.547	**0.578**	0.496	0.776	0.799	**0.801**	0.792

Table 1. Test results on 10,000 reviews (Base = always predict 5 stars, SVM = Linear SVM, NB = Naive Bayes). Best model per column in bold.

Model comparison. Naive Bayes had the best star accuracy on every target (for example 0.651 on overall, compared with 0.618 for the SVM and 0.560 for Ridge). The balanced SVM gives up some accuracy to handle rare ratings better, and it had a higher macro-F1 than Naive Bayes for rooms (0.463 vs 0.455) and location (0.312 vs 0.285). Most predictions are close to the true rating: within-1-star accuracy is between 0.89 and 0.97 for all models, and the confusion matrix (Figure 2) shows that most mistakes are between neighbouring ratings, like 4 and 5.

Baseline. Compared with the baseline, the best model improves star accuracy by 0.219 for overall, 0.115 for value, 0.135 for rooms, 0.081 for sleep quality and 0.067 for cleanliness, so for these it is really learning from the text. For **location** the improvement is only 0.008 (the SVM is actually 0.012 below the baseline), and the polarity baseline (0.876) is higher than every model (0.861 to 0.862). So we cannot say the models learned location sentiment beyond predicting the most common rating.

Aspect-trained vs overall-trained SVM (star accuracy on each aspect's own ratings):

Target	Aspect-trained	Overall-trained	Difference
Value	0.485	0.502	-0.017
Rooms	0.536	0.545	-0.009
Location	0.626	0.494	+0.132
Cleanliness	0.589	0.556	+0.033
Sleep quality	0.547	0.534	+0.014

Table 2. Training on an aspect's own ratings helps for location, cleanliness and sleep quality, but not for value and rooms. The location gain is mostly because the overall-trained model scores below the baseline there (0.494).

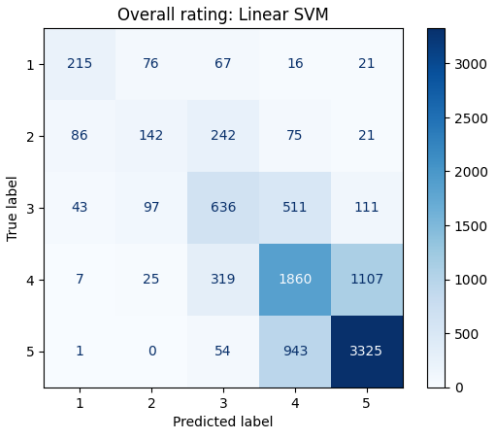


Figure 2. Confusion matrix for the overall rating, Linear SVM (rows = true stars, columns = predicted).

6. Conclusions

Simple counts of 1 to 3 word phrases with linear models predict ratings from review text much better than guessing for overall, value, rooms, cleanliness and sleep quality, and the predictions are usually within one star (95.6% for the overall SVM). Naive Bayes was our most accurate model. We kept the balanced SVM too because it handles rare low ratings better. Training on an aspect's own ratings helps most when the aspect has its own words, like cleanliness ("spotless", "dirty").

Limitations. Each model reads the whole review, so it cannot tell which sentence is about which aspect. In our demo, a review that said the location was "far from everything" still got a positive location score. Some of the top features are general sentiment words (for example "classy" for sleep quality), the 300,000-feature limit was reached, the ratings are skewed towards 5 stars, and our dataset has five aspects instead of the paper's seven, so our numbers cannot be compared directly with the paper's. We also did not tune the hyperparameters or use cross-validation.

Future work. Next we would assign each sentence to an aspect before scoring it, try class rebalancing or ordinal regression, and try transformer models like BERT.

References. [1] Y. Yu, *Aspect-based Sentiment Analysis on Hotel Reviews*, CS229 project report, Stanford University, 2016. [2] Hugging Face dataset jniimi/tripadvisor-review-rating (huggingface.co/datasets/jniimi/tripadvisor-review-rating), based on data from Li, Ott and Cardie (2013).